RAJALAKSHMI ENGINEERING COLLEGE

Rama Raghavi T 250701582-2025-CSE

R2

Started on	Tuesday, 15 September 2026, 12:15 AM
State	Finished
Completed on	Tuesday, 15 September 2026, 12:16 AM
Time taken	53 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Helper function to find the maximum of two numbers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;

    // Read the size of the chessboard
    if (scanf("%d", &n) != 1) return 0;

    // Allocate a 1D DP array to optimize space to O(n)
    int *dp = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int current_value;
            if (scanf("%d", &current_value) != 1) return 0;

            if (i == 0 && j == 0) {
                // Starting position
                dp[j] = current_value;
            } else if (i == 0) {
                // First row: can only come from the left
                dp[j] = dp[j - 1] + current_value;
            } else if (j == 0) {
                // First column: can only come from above
                dp[j] = dp[j] + current_value;
            } else {
                // General case: max of coming from above (dp[j]) or left (dp[j-1])
                dp[j] = current_value + max(dp[j], dp[j - 1]);
            }
        }
```

```
    }

    // The last element contains the maximum monetary value at (n-1, n-1)
    printf("%d\n", dp[n - 1]);

    // Free allocated memory
    free(dp);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course